

Scaling Across Knowledge Boundaries: A Case Study Of A Large-Scale Agile Software Development Project

Knut H. Rolland
Westerdals Oslo ACT & SINTEF ICT
Schweigaardsgt 14
0185 Oslo, Norway
+47 92882331
rolknu@westerdals.no

ABSTRACT

While, in practice, agile methods are increasingly used across large and distributed projects, the general applicability in large-scale development projects is still contested. Currently, there are only a few empirical studies of agile in the large, and especially accounts on how scaling is performed in practice are scarce. This paper aims at closing this gap by reporting from a case study of a large-scale agile effort in a highly prestigious IT-project in a Governmental organization in Norway. Theoretically, the paper draws from insights from studies of product innovation, and conceptualizes scaling in large-scale agile projects as a continuous process that involves establishing practices and artifacts for communicating and sharing across ‘knowledge boundaries’. Empirically, the paper contributes by illustrating how scaling is made possible and performed through a wide-range of novel practices and artifacts that emerge over time. We illustrate how these practices, roles and artifacts establish ways of *transferring*, *translating* and *transforming* knowledge across different teams and between different stakeholders. Our analysis shows that these practices challenge a widespread dichotomy between agile methods as being ‘flexible’ on one hand, and traditional methods being ‘rigid’ on the other hand. Instead, the tailored practices in the case both added structure and flexibility at different stages in the process.

CCS Concepts

• Software and its engineering -> Agile Software Development.

Keywords

Large-scale agile software development; Boundary work; Knowledge management; Case study

1. INTRODUCTION

In many contemporary organizations, agile software development has become the preferred approach for innovating new software and complex information systems. However, while initially

suggested as an approach prominently for small-scale projects, various approaches of agile are now being used in embedded systems [1], large-scale software development [2], and across distributed teams [3]. Thus, agile software development has now grown out of its infancy and is used for nearly all kinds of software development projects.

In this paper, we will concentrate on large-scale agile projects. While there is little agreement in the literature on what counts as ‘large-scale’, we pragmatically refer to projects involving more than two teams as a large-scale agile development project [4]. Potentially, however, large-scale agile also has a number of other characteristics. Typically large-scale project would involve a larger number of external experts in addition to team members [5]. This means that larger projects are likely to be increasingly multidisciplinary making it more difficult to share knowledge across involved stakeholders [6]. Larger projects are also likely to involve extensive integration with existing IT-systems in the customer organization and migration of large chunks data from legacy systems, which in turn implies more stakeholders [7]. All these aspects add to complexity of large-scale projects. Not only in terms of increasing complexity of the software being development itself, but also in terms of communicating with a wide array of different stakeholders [8]. Scaling agile the large the large thus involves multiple ‘dimensions’ [1, 7].

According to the empirical literature, some agile practices are found not to scale well. For instance, a study by Passivaara and colleagues found that the agile practice of ‘scrum-of-scrums’ did not work out as intended [9]. Interestingly, the study found that scrum-of-scrums involved so many stakeholders that it became difficult to know what to report that were of interests to the other teams. This empirical example is, however, arguably only one instance of a more deep-rooted and complex problem in large-scale agile [4]. Apart from Batra et al. [3] who argue that there is a need for balancing agile practices with more structured approaches in large projects, a more nuanced explanatory and conceptual framing is lacking in current literature. This is unfortunate since it gives both researchers and practitioners a limited set of concepts and understanding of the multifaceted complexity of large-scale agile software development. The question, then, is: *what practices are important for scaling agile in the large and how should scaling be conceptualized?*

To this end, this paper aims at contributing to proposing a conceptual framework focusing on the complex social and organizational aspects of sharing knowledge and coordinating in large-scale settings. Here we draw on Carlile’s [6, 10] framework of managing knowledge across contexts in order to shed light on ‘scaling’ agile in the large. A core concept in this body of work, is

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

XP '16 Workshops, May 24 2016, Edinburgh, Scotland UK
© 2016 ACM. ISBN 978-1-4503-4134-9/16/05...\$15.00
DOI: <http://dx.doi.org/10.1145/2962695.2962700>

‘knowledge boundaries’ that defines the *nature and complexity of the knowledge exchange needed*. This implies that people from different communities cannot just share information in a straightforward manner, but have to translate between communities and sometimes negotiate between communities in order to ‘share’ knowledge. Analytically, we use this insight in order to focus on the knowledge boundaries that need to be crossed when scaling up agile in large projects. Obviously, since large-scale software development to a larger extent than traditional product development, involves numerous interdependencies between multiple actors residing across different ‘knowledge boundaries’, Carlile’s insights becomes highly relevant.

Using these insights and concepts, we report from an ongoing case study of a large-scale agile project in a governmental organization. Empirically, we contribute by showing how the tailored agile practices are not just ad hoc adjustments of the standardized method, but *practices that make scaling across various knowledge boundaries possible*. Drawing on the concepts of Carlile [6, 10] we suggest that agile in the large need to tailor practices in order to scale across complex knowledge boundaries. However, we also see that these tailored practices challenge the widespread dichotomy between agile methods and practices as being flexible on one hand, and traditional methods adding stability and structure on the other hand [3]. The tailored practices in the case both added structure and flexibility at different stages in the process rejecting a dichotomy.

The rest of this short paper is structured as follows. In the next section, we present our interpretation and use of Carlile’s framework for conceptualizing scaling in large-scale agile. Then, in the third section, we present a case study of a large-scale agile project involving over 120 participants over 3 years from 2011 to late 2014. Because this is a short paper, the case presentation, methodological approach, and the results are presented here. And in the last section we briefly discuss our findings and conclude by giving some directions for further research.

2. CONCEPTUALIZING AGILE IN THE LARGE

Scaling agile in the large involves communication between various stakeholders, teams and different individuals. Arguably, for this to take place in an effective manner, the involved actors must share a minimum of common frames of reference and understanding. Henceforth, a key issue in scaling is to facilitate knowledge sharing across development teams, stakeholders, and other individuals involved in the project or related activities in the customer organization.

However, as conceptualized by Carlile [6, 10], this depends on the complexity of knowledge boundaries – or in other words, *the nature and complexity of the knowledge exchange needed*. Carlile reports from traditional product development and recognizes how different teams – or functions in a firm, over time, develop new knowledge that is locally embedded and situated in the work they carry out. They thus establish a ‘knowledge boundary’ or common frames and interpretations. This works well for communicating *within* teams, but simultaneously creates problems when working *across* teams since they do not necessarily share interpretations and the ‘investment’ in the specific type of knowledge [6, 10]. Increased novelty and innovation within teams is therefore likely to increase this ‘gap’ [10]. Accordingly, as described in the table below three different knowledge processes is needed to achieve this.

Drawing on this framework, we accomplish two things. First, the framework explains why agile in the large involving many teams, roles, stakeholders and experts become more complex because of the sheer amount of information, different ‘local’ invested knowledge, and different interests. Correspondingly, there is a need for need practices and a project infrastructure that supports the very different knowledge processes of transfer, translation and transformation. Secondly, the framework helps us methodologically, to analyse and generalize based on the case study data [11].

Table 1. Three types of knowledge boundaries, the corresponding knowledge processes, and their relevance for scaling. Drawing on [1].

Type of boundary	Knowledge processes	Relevance for scaling agile
Syntactic	Transferring knowledge through for example the use of information technologies, artifacts or documents having a predefined syntax or taxonomy.	The use of a common project IT infrastructure and tools like Jira in order to share huge amounts of information between large numbers of (possibly distributed) stakeholders.
Semantic	Translating knowledge between different communities’ local interpretations.	More than 2 development teams, multidisciplinary teams with testers, UX, architects and developers invokes the need to constantly translate between them.
Pragmatic	Transforming knowledge through negotiations with other communities.	The need for frequently negotiating and re-negotiating details of the solution that cut across different boundaries. Team A wants one specific architecture design, whereas Team B wants a conflicting design that better ‘fits’ their prior investments.

3. METHOD

A case study of a major software development was conducted from September 2014 to December 2015. The project, referred to as the Brownfield project, was a large-scale agile development effort involving over 120 participants over 3 years from 2011 to late 2014. The project was organized as four ‘scrum’ teams and one team loosely related to the project developing a business intelligence solution. The case can be considered a particularly ‘revelatory’ case study [11] because of the specific context. Firstly, the supplier, consulting company had just recently before starting on the Brownfield project been part of a prestigious large-scale agile software development project that was especially known nationally for being a success – and often used as a template for other large-scale agile projects in Norway. Secondly,

the customer had tried two times before earlier in the 2000s to implement the Brownfield project and failed considerably in both cases. Currently, there have been conducted 20 in-depth interviews with participants from both the Customer and the Consulting company. The interviews were from 50 minutes to 90 minutes long, and all except two interviews were transcribed. In addition, we have had numerous meetings, access to key documentation – including Jira and Confluence material, organized two workshops, and numerous seminars involving the Consulting company. The data has gone through one round of preliminary analysis involving thematic coding and theoretical coding using predefined categories drawn from the theoretical framework and additional emerging categories in the interviews. Triangulation was achieved by constantly comparing themes and statements across informants interviewed, participants in workshops, and documentation.

4. RESULTS

Initially, the scaling process in the Brownfield project was a failure. Both the consultants and the governmental organization confirmed this. The project had taken off with too many teams when it started in early 2011 on the first deliverable. During this first deliverable it failed to establish a working practices across teams and roles. The consequences of this situation were that the first deliverable there were numerous misunderstandings between the participants form the Governmental organization and the Consultant Company. In turn, this lead to a missed deadline, failing to put the first deliverable into actual use, and that the entire project was in danger of being closed down. Consequently, the project did not manage to scale during the first phase:

”...The supplier came in an focused on delivering on time – and that’s it. When, we suddenly understood that we were not prepared. They came in with sixty people and we were three architects and a few people functional responsible. It was hard – it was a rude awakening.” (Informant 13, customer)

However, the project was later in 2012 turned around during the second and third deliverables. Arguably, this was made possible through a series of novel practices and artifacts that made the project – slowly but surely, capable of scaling across the various complex knowledge boundaries. For example, temporally organized ‘*task forces*’ were established consisting of appointed developers from different teams to solve particularly pressing technical idiosyncrasies, as explained by a team leader:

“[We] had participants coming from the teams for helping IT operations to configure the build system (Jenkins) and to verify that things were working correctly...[]...like a task force.” (Informant 2).

A task force like this worked for about two of weeks. This exemplifies how this practice enables working across the complex boundary between IT operations and the development teams. Hence, making scaling of the project possible. Another novel practice was referred to as the ‘*ready-to-sprint*’ process. This was a comprehensive walk-through prior to sprints where team leaders, architects, product owner, and other expert roles came together in order to uncover important dependencies and establish a common understanding of what to do during next sprint. As one of the informant says:

“Many things were done [to improve the development process]. Specifically, the ready-to-sprint process that was introduced in order to avoid confusion of exactly what to deliver. It required

accepted user stories with acceptance criteria, and we demanded to say OK before we took them in.” (Informant 1).

This process could go on for several days involving everyone who was involved in the next sprint one way or another. Thus, the ‘ready-to-sprint’ process became a common way of scaling across different knowledge boundaries. In particular, this was important because of the many interdependencies between user stories, architecture, and resources. Other boundary practices and artifacts that were important for scaling are listed and briefly explained in the table below.

Table 2. Practices and artifacts that make working across knowledge boundaries possible.

Practices and artifacts	Boundaries scaled
Temporary task forces to solve common task across teams.	Scaling across teams and across project and IT operations through translating and transforming knowledge.
Champion roles focusing on practices related to specific technologies like java scripts.	Scaling across teams through translating knowledge about technologies and techniques.
‘Ready to sprint’ process involving all relevant stakeholders to identify challenging issues coming up in next sprint.	Scaling across teams, customer’s architect, product owner and architect roles of the consultant company through translating knowledge about detailed requirements.
Mini-demos within sprints.	Scaling through translating knowledge across individual developers and customers.
Architectural meeting.	Scaling through translating and transforming knowledge across architects in different teams and customer’s architects.
Alarms indicating that a build has been broken.	Scaling across different feature teams transferring knowledge about build being broken, and triggering <i>ad hoc</i> coordinating for translating knowledge across individual developers.
Prototyping features ahead of sprints in a deliverable.	Scaling across developers, UX designers, and customers through translating and transforming knowledge about scope, concept and features.

5. DISCUSSION AND CONCLUDING REMARKS

In this short paper we have highlighted some of the practices and artifacts found to facilitate scaling in a large-scale agile development effort. Hence, contrary to much existing emphasis, we see that to scale agile comprises much more than organizing development teams and just continuing with the ‘traditional’ agile practices as scrum-of-scrums, sprint planning in teams, and focus on collaboration within teams. Drawing on Carlile’s framework for managing knowledge across contexts [6, 10], *scaling can be conceptualized as a continuous process of transferring, translating and transforming knowledge across various communities and individuals*. Thus, this adds to existing literature that only focuses on various ‘dimensions’ that need to be scaled, and not on the actual practices of scaling themselves [1,7]. In so

doing, the paper also shows the potential of Carlile's framework for conceptualizing such scaling practices [6, 10].

Importantly, the novel practices emerging in the case became successful in scaling because they *facilitated knowledge translation* between different actors that tended to have different interpretations of for example requirements. Practices like 'mini demos' facilitated knowledge transformation in that users and developers learned from each other, and hence transformed their own knowledge. This was extremely important in a large-scale project since many small deviations from the customer's expectations at the main demo-session would make progress and hence learning in the project difficult. It is important to note that these practices do not comprise an alternative way of doing agile, but as an *addition* to other more traditional practices. At the same time, however, it seems that cohering too strictly to traditional Scrum and/or XP practices would not be enough to scale agile in the large. This confirms other studies arguing standard scrum/XP is not enough [2, 3], and that it needs tailoring in order to work in different contexts – as for example in the case of offshoring [13].

Compared to existing relevant literature this study adds explanatory power to what kind of practices would work for scaling and what will likely not work. In this light, scrum-of-scrum practice would in many settings be too complex to be successfully conducted since it would involve crossing too many knowledge boundaries at once and it does not recognize the need for mutual learning or translation. Especially, this can be the case if too many teams are added to the project simultaneously or too quickly. Scrum-of-scrums were practiced also in the Brownfield case but did not play a large role and did not work without the addition of the aforementioned practices. By and large, this confirms existing studies [2].

More over, the study shows that agile practices for scaling are not necessarily defined top-down by managers. Rather than being coherently planned by project managers, scaling seems to be better off by letting the actors involved find out what works and not. For example, coordination routines between architects working in different local teams, are best sorted out by the architects themselves. This is because they would recognize when the need to translate their knowledge and negotiate with the others will be necessary. Hence, this characteristic make such practices blend in nicely with other agile practices and are not necessarily in conflict with agile principles.

However, the results from the study also confirms recent studies [3] that the agile in the large need to both structure and agility [14]. On the other hand, as seen in the case structure was not in conflict with agility or vice versa. For example, the practice of 'ready-to-sprint' involved more structuring of user stories with acceptance criteria prior to a sprint. At the same time, in the long run, it also implied improved agility in terms of better distribution of work tasks among teams. Hence, our findings question the implicit assumption in much software development research that structured methods and agility are always opposing forces that need to be 'balanced'. Rather it *seems that structure sometimes is necessary for agility* – and vice versa. However, we acknowledge that this point needs more research.

One limitation of this study is obviously that we did not study the project as it was conducted – but in retrospect. This of course, is a limitation because informants may forget nuances and it becomes harder to reconstruct the chain of events. It is also harder to assess what different actors actually did in certain situations. However, to remedy this, we are planning to also conduct analysis of Jira data from the project.

To conclude, we would recommend future studies of large-scale agile projects needs to study project over longer periods of time. Studies need to include reconstructing the past as necessary context, but also following the project 'as it happens' over prolonged periods of time. Theoretically, studies of large-scale agile should also be more theoretically informed than existing studies, and the conceptualization used in this preliminary analysis of the case should be refined even more in future contributions.

6. ACKNOWLEDGMENTS

This article was written with support from the project Agile 2.0, which is supported by the Research council of Norway through grant 236759/O30, and by the companies Kantega, Kongsberg Defence & Aerospace, Sopra Steria and Sticos.

7. REFERENCES

- [1] Eklund, U, Olsson, H., and Strøm, N.J. 2014. Industrial challenges of scaling agile in mass-produced embedded systems. In T. Dingsøyr, N. B. Moe, R. Tonelli, S. Counsell, C. Gencel, and K. Petersen (eds). *Agile Methods: Large-Scale Development, Refactoring, Testing, and Estimation*. vol. 199, Springer Verlag, Heidelberg.
- [2] Paasivaara, M., and Lassenius, C. 2014. Communities of practice in a large distributed agile software development organization–Case Ericsson. *Information and Software Technology* 56, 12, 1556-1577.
- [3] Batra, D., Xia, W, VanderMeer, D. and Dutta, K. 2010. Balancing agile and structured development approaches to successfully manage large distributed software projects: A case study from the cruise line industry. *Communications of the Association for Information Systems* 27, 1, 379-394 .
- [4] Dingsøyr, Torgeir, and Nils Brede Moe. 2014. Towards Principles of Large-Scale Agile Development. In: T. Dingsøyr, N. B. Moe, R. Tonelli, S. Counsell, C. Gencel, and K. Petersen (eds). *Agile Methods. Large-Scale Development, Refactoring, Testing, and Estimation*, Springer, Heidelberg.
- [5] Rejab, Mawarny Md, James Noble, and Stuart Marshall: Coordinating Expertise Outside Agile Teams. 2015. In: Lassenius, C, Dingsøyr, T., and Paasivaara, M. (eds). *Agile Processes, in Software Engineering, and Extreme Programming*, pp. 141-153. Springer, Heidelberg.
- [6] Carlile, Paul R. 2002. A pragmatic view of knowledge and boundaries: Boundary objects in new product development. *Organization science* 13, 4, 442-455.
- [7] Ambler, Scott W. 2008. Agile software development at scale. In: *Balancing agility and formalism in software engineering*, pp. 1-12. Springer, Heidelberg.
- [8] Pikkarainen, Minna, Jukka Haikara, Outi Salo, Pekka Abrahamsson, and Jari Still. 2008. The impact of agile practices on communication in software development. *Empirical Software Engineering* 13, 3, 303-337.
- [9] Paasivaara, Maria, Casper Lassenius, and Ville T. Heikkilä. 2012. Inter-team coordination in large-scale globally distributed scrum: Do Scrum-of-Scrums really work?. In: *Empirical Software Engineering and Measurement (ESEM), 2012 ACM-IEEE International Symposium on*, pp. 235-238. IEEE.
- [10] Carlile, Paul R. 2004 Transferring, translating, and transforming: An integrative framework for managing

knowledge across boundaries. *Organization science* 15, 5, 555-568.

[11] Walsham, G.. 2006. Doing interpretive research. *European journal of information systems* 15, 3, 320-330.

[12] Yin, Robert K. 2003. *Case study research design and methods*, third edition., Sage, London.

[13] Bass, J. M. 2016. Large-Scale Offshore Agile Tailoring: Exploring Product and Service Organisations. In: Moe & Dingsøyr. *Large-scale agile development workshop XP 2016*, Edinburgh, UK.

[14] Conboy, K.. 2009. Agility from first principles: Reconstructing the concept of agility in information systems development. *Information Systems Research* 20, 3, 329-354.